EXPERIMENT 14

Write the Python program to implement the Alpha-Beta Pruning algorithm for gaming

Register Number: **192511013**

192511013

Aim

To write a Python program to implement Alpha-Beta Pruning, an improvement of the Minimax algorithm that gives the same result while skipping (pruning) branches of the game tree that cannot influence the final decision.

Algorithm

Step 1: Use the same game tree as in Minimax. Initialise alpha = -infinity (best value MAX can guarantee so far) and beta = +infinity (best value MIN can guarantee so far).

Step 2: If the current node is a leaf, return its utility value.

Step 3: MAX node: set best = -infinity. For each child, set best = max(best, value of child) and alpha = max(alpha, best). If beta <= alpha, stop examining the remaining children (beta cut-off).

Step 4: MIN node: set best = +infinity. For each child, set best = min(best, value of child) and beta = min(beta, best). If beta <= alpha, stop examining the remaining children (alpha cut-off).

Step 5: Pass the current alpha and beta values down to every child call.

Step 6: Return best to the parent. The value at the root is the same as the Minimax value.

Step 7: Print the optimal value, the number of leaves evaluated, the order of evaluation and the leaves that were pruned.

192511013

Program

```
import math

# Same game tree as the Minimax experiment
leaves = [3, 5, 6, 9, 1, 2, 0, -1]
DEPTH = 3
evaluated = []

def alpha_beta(depth, index, is_max, alpha, beta):
    if depth == DEPTH:
        evaluated.append(leaves[index])
        return leaves[index]
    if is_max:
        best = -math.inf
        for child in (index * 2, index * 2 + 1):
            best = max(best, alpha_beta(depth + 1, child, False, alpha, beta))
            alpha = max(alpha, best)
            if beta <= alpha:               # beta cut-off
                break
        return best
    else:
        best = math.inf
        for child in (index * 2, index * 2 + 1):
            best = min(best, alpha_beta(depth + 1, child, True, alpha, beta))
            beta = min(beta, best)
            if beta <= alpha:               # alpha cut-off
                break
        return best

if __name__ == "__main__":
    print("Leaf values:", leaves)
    value = alpha_beta(0, 0, True, -math.inf, math.inf)
    print("\nOptimal value of the root (MAX player) =", value)
    print("Leaf nodes evaluated :", len(evaluated), "of", len(leaves))
    print("Order of evaluation  :", evaluated)
    pruned = [v for v in leaves if v not in evaluated]
    print("Leaf nodes pruned    :", pruned, f"({len(pruned)} leaves skipped)")
```

192511013

Output

```
Leaf values: [3, 5, 6, 9, 1, 2, 0, -1]

Optimal value of the root (MAX player) = 5
Leaf nodes evaluated : 5 of 8
Order of evaluation  : [3, 5, 6, 1, 2]
Leaf nodes pruned    : [9, 0, -1] (3 leaves skipped)
```

192511013